BÖLÜM 12 / MODULE 12

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI

RESTful API Design, Richardson Maturity Model & OpenAPI

REST prensipleri, HTTP durum kodları, kaynak modelleme, HATEOAS, API versiyonlama, idempotency ve OpenAPI / Swagger dokümantasyonu.

Resource modeling, idempotency semantics, HTTP status codes, Richardson Maturity Model level 3, versioning, and OpenAPI contracts.

111

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Temel Kavramlar, Tarihsel Evrim ve Temel Mühendislik Çerçevesi

RESTful API Design, Richardson Maturity Model & OpenAPI - Core Concepts, Historical Evolution & Foundational Frameworks

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson01;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

112

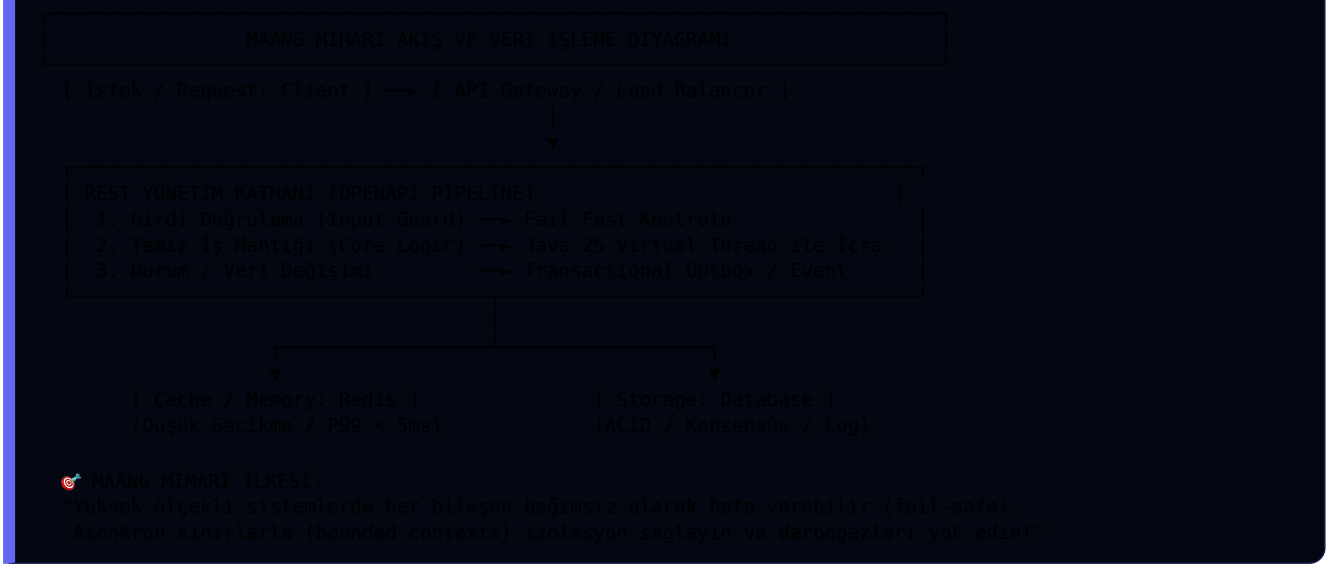
BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Mimari Bileşenler, Çalışma Mekanizması ve Veri Akışı

RESTful API Design, Richardson Maturity Model & OpenAPI - Architectural Components, Internal Mechanics & Data Flow

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson02;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

113

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Tasarım İlkeleri, Sözleşmeler ve Temiz Arayüz Tanımları

RESTful API Design, Richardson Maturity Model & OpenAPI - Design Invariants, Contracts & Clean Interface Abstractions

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson03;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

114

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Yaygın Anti-Pattern'ler, Tasarım Hataları ve Önleme Stratejileri

RESTful API Design, Richardson Maturity Model & OpenAPI - Common Anti-Patterns, Design Pitfalls & Prevention Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson04;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

115

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Yüksek Performans, Optimizasyon ve Bellek / CPU Yönetimi

RESTful API Design, Richardson Maturity Model & OpenAPI - High Performance, Optimization & Memory/CPU Resource Management

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson05;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

116

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Eşzamanlılık, Thread Güvenliği ve Dağıtık Sistem Etkileri

RESTful API Design, Richardson Maturity Model & OpenAPI - Concurrency, Thread-Safety & Distributed Systems Implications

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson06;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

117

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Hata Yönetimi, Dayanıklılık (Resilience) ve Geri Düşme (Fallback)

RESTful API Design, Richardson Maturity Model & OpenAPI - Fault Handling, Resilience Patterns & Graceful Fallback Strategies

⚡ Mimari Akış ve Süreç Şeması ⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson07;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

118

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Test Stratejileri, Test Edilebilirlik ve Doğrulama Hatları

RESTful API Design, Richardson Maturity Model & OpenAPI - Testing Strategies, Testability & Automated Verification Pipelines

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson08;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

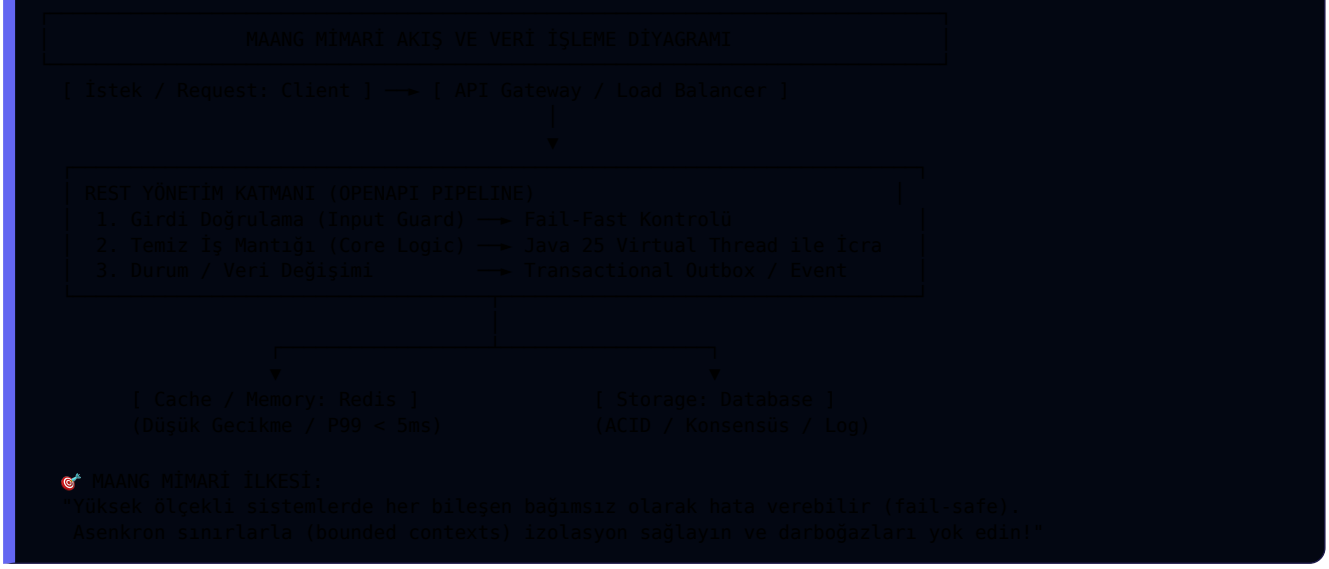
119

BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

RESTful API Design, Richardson Maturity Model & OpenAPI - Observability, Telemetry, Tracing & Production Troubleshooting

⚡ Mimari Akış ve Süreç Şeması ⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

120

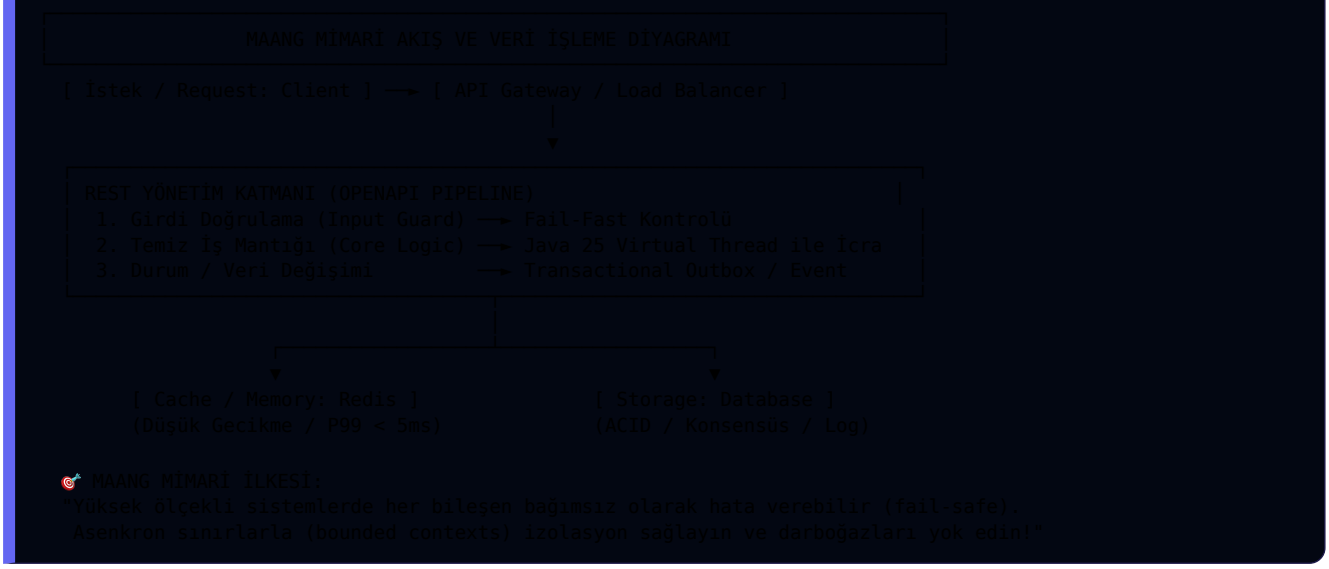
BÖLÜM: RESTFUL API TASARIMI, RICHARDSON OLGUNLUK MODELI & OPENAPI | MODULE: RESTFUL API DESIGN, RICHARDSON MATURITY MODEL & OPENAPI

RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI - MAANG Mülakat Ustalığı, Mimari Ödünleşimler (Trade-Offs) & Staff+ Vizyonu

RESTful API Design, Richardson Maturity Model & OpenAPI - MAANG Interview Mastery, Architectural Trade-Offs & Staff+ Vizyon

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (REST mikroservisleri) ve otomatik yönlendirme robotları (OpenAPI akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (REST microservices), and dynamic routing switches (OpenAPI pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde RESTful API Tasarımı, Richardson Olgunluk Modeli & OpenAPI Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde REST mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): OpenAPI katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for RESTful API Design, Richardson Maturity Model & OpenAPI: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), REST implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in OpenAPI don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: REST & OpenAPI Kurumsal Uygulaması
package com.maang.se.module12.lesson10;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class RESTArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record RESTRequest(String transactionId, String payload, Instant timestamp) {
        public RESTRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record RESTResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class RESTExecutionService {
        public RESTResult processRequest(RESTRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new RESTResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering REST Engine Başlatıldı...");
        var service = new RESTExecutionService();
        var req = new RESTRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

REST kurgulanırken OpenAPI bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling REST logic directly to synchronous blocking calls into OpenAPI is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• REST mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design REST components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.